



Working on it ⓘ

Posted on March 24, 2022 Last update June 26, 2023

I finally discovered why Javascript Sets exist!

My first practical takeaway from learning Clojure

I've used [Javascript Sets](#) to de-duplicate items in an array for a few years, but I never understood it beyond that. It felt like one of those obscure language features that would never fully make it into my programs...

```
// My only usage of sets so far  
const uniqueTags = Array.from(new Set(tags))
```

But reading [Russ Olsen's Getting Clojure](#) (which is super well written and fun, by the way!), I casually came upon this line:

Since sets are all about membership, the main thing you can do with them is discover if this or that value is in the set.

(Page 32)

And then it dawned on me: Sets are *solely* a collection of unique values, so there's no better tool for these use cases it specializes in.

Use-case #1: checking if item is in collection

`Set.has` is faster than `Array.includes`:

```
// With Arrays
const singletonTypes = ['home', 'settings']
const isSingleton = singletonTypes.includes(doc._type)

// With Sets - faster
const singletonTypes = new Set(['home', 'settings'])
const isSingleton = singletonTypes.has(doc._type)
```

Of course, with few items this is negligible. But this could be a performance opportunity when there are hundreds, thousands of items in a set that is being checked multiple times per second, such as the render method of hundreds of a React/Svelte components.

Use-case #2: ensuring collection has no duplicates

This is way easier with sets:

```
// With arrays, you manually need to check if item isn't already there
if (!singletonTypes.includes('settings')) {
  singletonTypes.push('settings')
}

// With Sets
singletonTypes.add('settings') // if already exists, it'll be skipped
```